

Manual de Android Moderno con Kotlin y Jetpack Compose

Introducción

El desarrollo de aplicaciones Android ha evolucionado significativamente durante los últimos años. Tradicionalmente, las interfaces gráficas se construían utilizando archivos XML y componentes clásicos como Activities, Fragments, ListViews y RecyclerViews. Sin embargo, Google introdujo Jetpack Compose como el nuevo paradigma oficial para la creación de interfaces modernas.

Jetpack Compose permite construir aplicaciones mediante código Kotlin, eliminando gran parte de la complejidad asociada al desarrollo tradicional basado en XML. Gracias a este enfoque declarativo, el desarrollo resulta más rápido, mantenible y alineado con tecnologías modernas como Flutter y React.

Este manual resume los principales conceptos estudiados durante el curso, mostrando el proceso de creación de interfaces modernas mediante componentes reutilizables llamados Composables.

Objetivos

Objetivo General

Aprender los fundamentos del desarrollo de aplicaciones Android modernas utilizando Kotlin y Jetpack Compose.

Objetivos Específicos

- Comprender la arquitectura básica de Android Studio.
- Crear interfaces mediante Composables.
- Implementar componentes visuales modernos.
- Manejar estados reactivos.
- Utilizar corrutinas para operaciones asíncronas.
- Implementar listas eficientes mediante LazyColumn y LazyRow.
- Aplicar buenas prácticas de diseño utilizando Material Design 3.
- Comprender el funcionamiento de Scaffold como estructura principal de la aplicación.

Requisitos Previos

Para desarrollar aplicaciones Android modernas es necesario contar con:

Software

- Android Studio
- JDK (Java Development Kit)
- SDK de Android
- Kotlin
- Jetpack Compose

Conocimientos

- Programación básica en Kotlin.
- Funciones.
- Variables.
- Estructuras de control.
- Conceptos básicos de programación orientada a objetos.

Android Moderno y Jetpack Compose

Jetpack Compose es el framework oficial de Google para la construcción de interfaces gráficas.

La principal diferencia respecto al desarrollo tradicional es que ya no se requiere construir la interfaz mediante XML.

Antes:

```
<Button
    android:text="Aceptar"/>
```

Ahora:

```
Button(
    onClick = { }
) {
    Text("Aceptar")
}
```

Todo se construye mediante funciones anotadas con:

@Composable

Estas funciones representan componentes reutilizables de la interfaz.

Composables

Un Composable es una función especial que genera elementos visuales.

Ejemplo:

```
@Composable
fun Saludo() {
    Text("Hola Mundo")
}
```

Ventajas:

- Reutilización.
- Mantenimiento sencillo.
- Menor cantidad de código.
- Interfaces declarativas.

Botones

Durante el curso se estudiaron diversos tipos de botones.

Button

Botón estándar.

```
Button(
    onClick = {}
) {
    Text("Guardar")
}
```

Floating Action Button

Botón flotante utilizado para acciones principales.

```
FloatingActionButton(
    onClick = {}
) {
    Icon(Icons.Default.Add, null)
}
```

Small Floating Action Button

Versión compacta.

```
SmallFloatingActionButton(  
    onClick = {}  
) {  
    Icon(Icons.Default.Add, null)  
}
```

Large Floating Action Button

Versión ampliada.

```
LargeFloatingActionButton(  
    onClick = {}  
) {  
    Icon(Icons.Default.Add, null)  
}
```

Extended Floating Action Button

Permite mostrar texto e ícono.

```
ExtendedFloatingActionButton(  
    text = { Text("Agregar") },  
    icon = { Icon(Icons.Default.Add, null) },  
    onClick = {}  
)
```

Personalización mediante Shape

Los botones pueden personalizarse utilizando diferentes formas.

Ejemplo:

```
shape = RoundedCornerShape(16.dp)
```

o

```
shape = CutCornerShape(12.dp)
```

Esto permite adaptar la apariencia de la interfaz.

Cards

Las Cards permiten agrupar información visualmente.

Ejemplo:

```
Card {  
    Column {  
        Text("Título")  
        Text("Descripción")  
    }  
}
```

Aplicaciones:

- Productos.
- Noticias.
- Publicaciones.
- Usuarios.

Listados

Los listados son uno de los componentes más utilizados.

Column

Renderiza todos los elementos simultáneamente.

```
Column {  
    items.forEach {  
        Text(it)  
    }  
}
```

Adecuado para pocos elementos.

Problema de Column

Si existen miles de elementos:

5000 registros

Todos serán renderizados al mismo tiempo.

Esto consume:

- Memoria.
- CPU.
- Batería.

LazyColumn

Solución moderna equivalente a RecyclerView.

```
LazyColumn {  
    items(lista) {  
        Text(it)  
    }  
}
```

Ventajas:

- Renderiza únicamente lo visible.
- Reutiliza componentes.
- Mejor rendimiento.

LazyRow

Versión horizontal.

```
LazyRow {  
    items(lista) {  
        Text(it)  
    }  
}
```

LazyVerticalGrid

Permite mostrar elementos en formato cuadrícula.

```
LazyVerticalGrid(  
    columns = GridCells.Adaptive(128.dp)  
)
```

Ideal para:

- Catálogos.
- Galerías.
- Tiendas virtuales.

Scaffold

Scaffold es la estructura principal de una aplicación Compose.

Proporciona:

- TopBar.
- BottomBar.
- Snackbar.
- FloatingActionButton.
- Drawer.
- Área de contenido.

Ejemplo:

```
Scaffold(  
    topBar = {},  
    floatingActionButton = {}  
) { padding ->  
}
```

Snackbar

Permite mostrar mensajes temporales.

Ejemplo:

```
SnackbarHostState()
```

Uso típico:

- Confirmaciones.
- Notificaciones.
- Acciones reversibles.

Ejemplo práctico:

Correo enviado
DESHACER

Corrutinas

Las corrutinas permiten ejecutar operaciones asíncronas.

Ejemplo:

```
scope.launch {  
  
}
```

Se utilizan cuando:

- Hay animaciones.
- Consultas de red.
- Operaciones de larga duración.
- Snackbars.
- Bottom Sheets.
- Drawers.

Logcat

Logcat permite visualizar información de ejecución.

Ejemplo:

```
Log.d("MyApp", "Mensaje")
```

Tipos principales:

Tipo	Uso
Verbose	Información detallada

Debug	Depuración
Info	Información general
Warning	Advertencias
Error	Errores
	Fallos críticos

Bottom Sheet

Panel emergente que aparece desde la parte inferior.

Ejemplo:

`ModalBottomSheet()`

Características:

- Animaciones integradas.
- Estado controlado.
- Interacción modal.

Aplicaciones:

- Filtros.
- Formularios rápidos.
- Menús contextuales.

Navigation Drawer

Menú lateral deslizable.

Ejemplo:

`ModalNavigationDrawer()`

Permite:

- Navegación principal.
- Acceso a secciones.
- Organización de funcionalidades.

Generalmente incluye:

- Inicio.
- Perfil.
- Configuración.

- Ayuda.

Manejo de Estado

Compose utiliza estados reactivos.

Ejemplo:

```
var mostrar by remember {  
    mutableStateOf(false)  
}
```

Cuando cambia el estado:

```
mostrar = true
```

La interfaz se actualiza automáticamente.

Buenas Prácticas Aprendidas

Utilizar Jetpack Compose

Evitar XML cuando sea posible.

Reutilizar Composables

Crear componentes reutilizables.

Utilizar LazyColumn

Para listas grandes.

Manejar estados correctamente

Usar:

```
remember  
mutableStateOf
```

Utilizar Corrutinas

Para operaciones asíncronas.

Aplicar Material Design 3

Mantener consistencia visual.

Evitar renderizados innecesarios

Utilizar componentes Lazy.

Organizar el proyecto

Separar componentes por funcionalidad.

Ejemplo:

```
ui/  
├── buttons  
├── cards  
├── snackbar  
├── drawer  
├── bottomsheet  
└── lists
```

APIs y Paquetes Utilizados

Principales paquetes observados durante el curso:

```
androidx.compose.material3  
androidx.compose.foundation  
androidx.compose.ui  
kotlinx.coroutines  
android.util.Log
```

Conclusiones

Jetpack Compose representa la evolución natural del desarrollo Android moderno. Gracias a su enfoque declarativo basado en Kotlin, permite crear interfaces más limpias, mantenibles y eficientes que las desarrolladas mediante XML tradicional.

Durante el curso se aprendieron los principales componentes visuales, técnicas de manejo de estado, corrutinas, navegación y optimización de listas mediante estructuras Lazy, proporcionando una base sólida para desarrollar aplicaciones Android modernas siguiendo las recomendaciones actuales de Google.

Bibliografia

<https://github.com/Celestegvr/CURSOJETPACK.git>